

Lab 2 - MPI I/O

Large Scale Hybrid Computing

University of Minho

The Lab 2 focus on the development of scalable parallel code using MPI's input and output primitives. Use a Deucalion node with ARM CPUs (by specifying the partition `normal-arm` when submitting a job). Compile the code either in the frontend or on the compute nodes (the latter is advised).

This lab tutorial includes five exercises to be solved during the lab class. Copy the source files, as usual, from `/projects/F202500010HPCVLABUMINHO/LSHC/lab2` to your home directory in Deucalion:

```
cp -r /projects/F202500010HPCVLABUMINHO/LSHC/lab2 .
```

To load the compiler in the environment use the following commands (do not forget to also use them in your submission script):

GNU Compiler: `module load GCC/14.3.0.`

MPI Libraries: `module load OpenMPI/5.0.8-GCC-14.3.0.`

You can use other GNU/OpenMPI combinations at your own risk, but ensure that OpenMPI is compiled with the correct version of the compilation toolchain.

1. Analyse and profile the codebase

Goals: to understand the limitations of POSIX I/O primitives.

Lab 1.1 Consider the POSIX-based I/O version of the provided MPI code. It contains two I/O patterns, where processes write to a single shared file or to individual files, according to the schematic in figure 1. Analyse and complete the code, considering the hints on the comments.

Lab 1.2 Execute and plot the time required to run each of the I/O patterns using 1, 12, 24, 36, and 48 processes. Use a spreadsheet to plot and visualise the results. How does the increase in the number of processes affect performance? Which of the patterns performs and scales best?

Lab 1.3 Adapt each I/O function to output a total of 8 GiB of dummy data, automatically split among the number of processes used. Execute and plot the time required to run each of the I/O patterns using 1, 12, 24, 36, and 48 processes, using the same spreadsheet to compare the results.

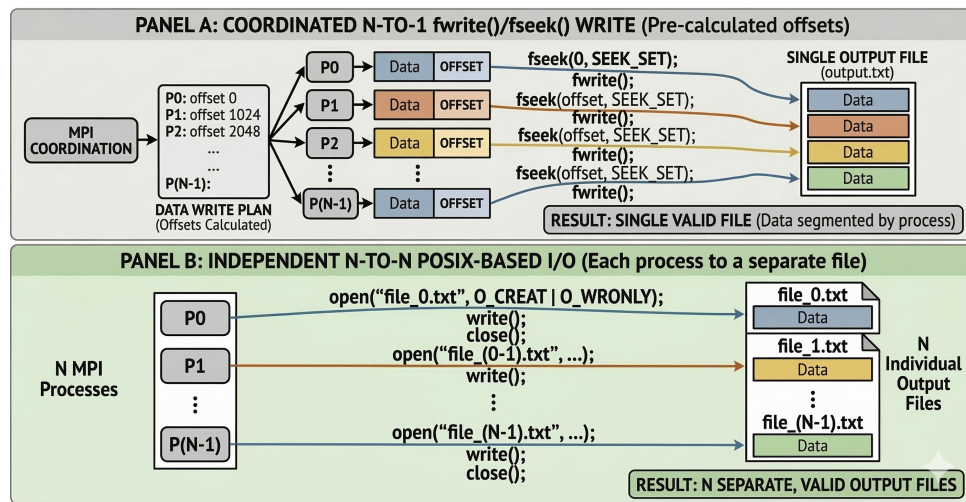


Figure 1: N-to-1 and N-to-N I/O patterns (thanks nano Banana!)

Please remove/overwrite the file after each execution.

2. MPI-based I/O

Goals: to develop skills in developing code with MPI's I/O primitives.

Lab 2.1 Consider the provided MPI code skeleton. It contains two I/O functions to write to a shared file, one where MPI calculates the offsets and another where they are calculated manually. Complete the code sections highlighted in the file with the primitives adequate for each phase of the communication, according to the comments.

Lab 2.2 Repeat the tests and measurements performed in exercises 1.2 and 1.3 with this codebase. Plot the MPI/POSIX speedups of the original data write and the 8 GiB data. How does the MPI scale with the number of processes used? How does it compare to the use of POSIX primitives for both datasets?

Please remove/overwrite the file after each execution.

Lab 2.3 Execute and measure the MPI I/O codebase, without manual offset calculation, with 8 processes per node, using 16 nodes, writing a total of 16 GiB of data. Vary the number of aggregators, using 1, 4, 8, and 16, by modifying the info argument:

```
MPI_Info info;
MPI_Info_create(&info);
MPI_Info_set(info, "cb_nodes", "2"); // adapt "2"
```

How does the number of aggregators affect performance? Repeat the test for the POSIX-based N-to-1 pattern implementation, and compare the results. How big is the performance penalty of using POSIX?

3. /tmp/ by I/O Performance

Goals: to dip your toes in different filesystems.

Lab 3.1 Consider the measurements performed in exercise 2.2. Rerun those tests but write the file in the node's storage, by specifying the /tmp/ directory. Plot the /tmp/ to Lustre speedup for 48 processes and quantify the improvements in performance.